Lab: Fuzzing Practice in Pharo

Duration: 1 hours **Tool:** Pharo

In this practice, we will learn how to use fuzz testing to discover bugs in a library. In its simpler form, fuzzing is about generating random input strings to feed to a library. With bare strings, we can imagine fuzzing a library implementing a parser. For this purpose, we will use the phuzzer library on top of Pharo 12.

Setup

For this practice, you will need to:

- install the pharo launcher
- install Pharo12 through the launcher
- install phuzzer in Pharo:
  - open a playground
  - execute the following expression:

```
Metacello new
  baseline: 'Phuzzer';
  repository: 'github://alamvic/phuzzer:main';
  onConflictUseIncoming;
  load.
```

Fuzzing some strings

Most of this practice will happen in the playground. Be sure to get used to the shortcuts, the `Do it and Go` command, and the inspect.

First, let's fuzz a string with the following code:

```
fuzzer := PzRandomFuzzer new.
fuzzer fuzz
```

- Execute the code above a couple of times and check the results. Are they always the same?

`PzRandomFuzzer` will generate random strings that have random size and that contain characters in a range `[start,start+range]`. We can control the string generation with the following parameters: - `minLength:` : the minimum length of the generated string - `maxLength:` : the maximum length of the generated string - `charStart:` : the start unicode codepoint for the character range - `charRange:` : the number of codepoints that the character range spans

Try configuring the random fuzzer to do the following:

- generate only numbers
- generate very long strings
- generate short alphanumeric (only letters) strings

Things to think about:

- Think about the limits of this fuzzer? How "complex" are the strings we can generate?
- Extra: How could you modify this fuzzer to generate hexadecimal strings? Tip: you may want to create your own subclass or change the code of the `PzRandomFuzzer` class.

Fuzzing a Date parser

In Pharo, we have a method `asDate` in the String class, that parses a string in `m-d-yyyy` format and gives us a date. Try the following code:

```
'11-2-1992' asDate
```

Let's use our fuzzer to generate random inputs to play with this parser. In the following piece of code, we use together a fuzzer and a runner. The runner takes care of running a block of code (`[ :e | e asDate ]`) for each input and analysing the results. The simple runner categorises the results as follows: - **an error:** if the block fails with an unexpected exceptions - **a success:** if the block fails with an expected exception or if it does not fail at all

Run the code below which fuzzes 20 times and inspect the results

```
f := PzRandomFuzzer new.
r := PzBlockRunner on: [ :e | e asDate ].
f run: r times: 20.
```

Things to think about:

- Check the successes. Are they real successes?
- What about the errors? Are there some of them that look like bugs? How can you make the difference between bugs and expected behavior?
- What would happen if our fuzzer only generates strings with numbers? Does it change the results whether those strings are short or long?
- What about fuzzing libraries with more than one parameter? What if we need one parameter depending on another one?

Tips: Remember you can use `select: aBooleanBlock` on collections to filter the results and help you analyse results.

Refining the results with knowledge

The `asDate` method throws, in some cases, a `DateError` when parsing invalid dates. The `PzBlockRunner` supports an `expectedException` and will classsify those as `PASS-FAIL`. The code below does the same fuzzing as before, but declaring `DateError` as an expected exception.

```
f := PzRandomFuzzer new.
r := PzBlockRunner on: [ :e | e asDate ].
r expectedException: DateError.
f run: r times: 20.
```

Fuzz for 1000 times. How many errors did you get? Repeat the experiment several times. Are the number of errors around the same number? How could you validate that?

Tip: collections in Pharo understand messages like `count: aBooleanBlock`, `average`, `stdev`.

Things to think about:

- We clearly need a way to distinguish between correct and incorrect outputs. Could you think on a smart way to do it without duplicating the date parser logic?

Writing a Grammar

Random strings just got us so far. A more advanced way to fuzz is to generate random strings that follow some structure. For example, we could generate strings that look like dates such as `2-2-2` or `20/20/23`. We can do this with grammars.

Create a grammar class for MDYYYY dates called `MyDateGrammar` as follows:

```
GncBaseGrammar subclass: #MyDateGrammar
    instanceVariableNames: 'ntDate ntDay ntSeparator ntMonth
ntYear'
    classVariableNames: ''
    package: 'MyFuzzer'
```

And then, add the method `defineGrammar` that gives strings that look like dates. For example, the grammar below builds dates that look like `[number]-[number]-[number]`

```
MyDateGrammar >> defineGrammar

  "The superclass defines how to generate numbers"
    super defineGrammar.

    ntDate --> ntDay , ntSeparator , ntMonth , ntSeparator ,
ntYear.
    ntSeparator --> '-'.
    ntDay --> ntNumber.
    ntMonth --> ntNumber.
    ntYear --> ntNumber.
    ^ ntDate
```

Then we can use that grammar to build a fuzzer as follows

```
(PzGrammarFuzzer on: MyDateGrammar new) fuzz
```

Try the fuzzer above to fuzz our date parser. Do you find new bugs?

Things to think about:

- Now we have things that look like dates, but some of those dates are strange. Are

there successes that are not so succeses?
- We went from entirely random strings to almost non-random strings. How could we do better? Could you think of alternative strategies? Something middle-ground?

More exercises

Check the documentation of *gnocco* the grammar library in https://github.com/Alamvic/gnocco/.

1. Modify the date grammar or build new ones implementing different variants:
    - the day or month fields have no more than two digits
    - what would it take to have names as months (e.g., february, january)?
    - what if we would like to have more field separators (e.g., /, -, spaces)
2. Try fuzzing the `Color class >> fromString:` method. What kind of inputs do you need to generate? Check the implementation and see what could be an effective grammar for this method.
3. Download the XML Parser library and fuzz it. XML is a super well tested library, so we could expect to have few bugs here. Could you run first mutation testing on it to find weak parts of the API, then fuzz it?

Further?

A natural next step for this practice is to introduce mutation fuzzing. Mutation fuzzing is a fuzzer generates inputs by modifying (mutating) values. We can then combine grammar fuzzing with mutation fuzzing: grammars generate structured fuzzing, mutations take those and mutate them into date wanabees. Those probably have high changes of find subtle bugs.

Later these can be combined with code coverage, trying to generate inputs that cover new paths in the program.